

摘要

边缘端目标检测要求在有限功耗和有限片上资源下完成较低延迟的卷积神经网络推理。FPGA 与 MPSoC 平台具有可定制数据通路、丰富片上存储和灵活软硬件协同的优势, 适合部署轻量级目标检测网络。然而, 在真实网络端到端运行中, 加速器性能并不只取决于乘加阵列规模; DMA 启动、输入特征图重排、权重加载、部分和反馈、多轮 K pass 同步以及软硬件调度空泡都会显著降低阵列有效利用率。

本文面向 Xilinx Kria KV260 / XCK26 平台, 设计并实现了一种用于 YOLOv3-tiny 单尺度链式推理的 INT8 卷积加速器。硬件采用参数化 systolic array 作为卷积核心, 支持 3x3 卷积、原生 1x1 卷积、INT32 部分和累加、逐通道重量化、查表激活、池化以及 AXI-Lite/AXI-Stream 系统接口。软件侧基于 Vitis bare-metal runtime 负责层描述、空间 tile 调度、DMA 启动、量化参数配置、输出重排和 YOLO decode, 从而形成从 DDR 图像输入到 Conv0–Conv9 输出解析的完整 PS/PL 协同链路。

围绕真实 YOLO 层的非计算开销, 本文进一步提出并实现了多级数据流与调度优化。首先, 通过 batch stream 和预打包权重布局减少每个 tile 内的 DMA 启动与运行时重排开销; 其次, 引入 raw-HWC tile cache, 使软件能够传输自然 HWC tile, 由硬件在片上完成 1x1/3x3 向量重放, 并在后端层使用 full-tile 调度降低空间 tile 启动开销; 再次, 针对多 K pass 卷积中的 partial-PSUM 反馈瓶颈, 实现了流水化 drain、early drain、continuous PSUM collector 和 pass-level prefetch, 使权重准备、IFM replay、部分和反馈与当前 pass 计算尽可能重叠。本文同时设计了阶段、子阶段、pass 级和列级性能计数器, 用于解释瓶颈迁移和验证优化有效性。

实验结果表明, 所设计系统在 KV260 上完成 Conv0–Conv9 batch-chain bit-exact 验证, 并在两张 DDR 动态图像上得到稳定的 YOLO decode 结果。最终推荐实现采用 Vivado/Vitis 2022.2、100 MHz 时钟、ROWS=18、COLS=8、COUT_TILE=16 的配置, 在 D:/MPSoC/b_kprefetch_22 构建中完成布局布线, 资源占用为 84480 个 CLB LUT、53260 个 CLB Register、63 个 BRAM Tile、24 个 URAM 和 184 个 DSP。两张图像的端到端 DDR demo 延迟约为 288 ms。该结果并不定位为实时视频性能领先, 而是证明在资源受限 MPSoC 上, 围绕数据流、部分和反馈和 pass 级调度的系统优化能够将早期约 1.18 s 的链式推理延迟降低到约 0.29 s, 并给出可解释的瓶颈分析路径。

关键词: YOLOv3-tiny, MPSoC, FPGA 加速器, systolic array, 数据流优化, partial-PSUM, INT8 量化

目录

第一章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 本文主要工作	2
1.4 论文组织结构	2
第二章 背景与相关工作	5
2.1 卷积计算与分块表示	5
2.2 INT8 量化与后处理	6
2.3 Systolic Array 与数据流	6
2.4 YOLOv3-tiny 单尺度网络	6
2.5 MPSoC 平台与 AXI 接口	7
2.6 相关工作比较	7
第三章 加速器系统架构	9
3.1 系统总体结构	9
3.2 PL 加速器内部数据流	9
3.3 Systolic 计算核心	10
3.4 后处理流水线	10
3.5 AXI-Lite 配置与性能计数器	10
3.6 软件运行时协同	11
第四章 数据流与分块调度优化	13
4.1 三维分块调度	13
4.2 Batch Stream	14
4.3 预打包权重布局	14
4.4 Raw-HWC Tile Cache	14
4.5 Backend Full-Tile 调度	15
4.6 数据流优化小结	15
第五章 Partial-PSUM 反馈与系统集成	17
5.1 Partial-PSUM 反馈语义	17
5.2 流水化 PSUM Drain	17

5.3	Early Drain 与 Continuous PSUM Collector.....	18
5.4	Pass-Level Prefetch.....	18
5.5	STREAM_CFG 与调度开关.....	19
5.6	性能计数器设计.....	19
5.7	探索性 Column-Level PSUM.....	19
5.8	本章小结.....	20
第六章	验证与实验结果.....	21
6.1	实验设置.....	21
6.2	正确性验证.....	21
6.3	端到端性能与优化消融.....	22
6.4	阶段计数器分析.....	23
6.5	资源与时序.....	23
6.6	与相关工作的定性比较.....	24
6.7	局限性与后续工作.....	25
6.8	本章小结.....	25
参考文献		27

表格索引

表 2.1	Conv0–Conv9 单尺度链路层参数	7
表 5.1	主要性能计数器类别	19
表 6.1	关键正确性验证结果	22
表 6.2	端到端 DDR demo 优化消融	22
表 6.3	最终 DDR image demo 结果	22
表 6.4	最终版本关键阶段计数器	23
表 6.5	最终推荐构建资源与时序	24
表 6.6	与代表性工作的定性比较	24

插图索引

图 3.1	MPSoC 端到端系统结构.....	9
图 3.2	PL 加速器内部数据流.....	10
图 4.1	K/Cout/spatial 三维调度关系	13
图 4.2	prepacked IFM stream 与 raw-HWC tile cache 对比	15
图 5.1	Partial-PSUM 反馈与 pass-level prefetch 时间线	18

主要符号对照表

CNN	卷积神经网络 (Convolutional Neural Network)
IFM	输入特征图 (Input Feature Map)
OFM	输出特征图 (Output Feature Map)
PSUM	部分和 (Partial Sum)
DSP	数字信号处理单元 (Digital Signal Processing slice)
BRAM	块随机存取存储器 (Block RAM)
LUT	查找表 (Look-Up Table)
PE	处理单元 (Processing Element)
FSM	有限状态机 (Finite State Machine)
AXI	高级可扩展接口 (Advanced eXtensible Interface)
DMA	直接存储器访问 (Direct Memory Access)
K_{total}	展开后输入通道总维度, $K_{\text{total}} = C_{in} \times K_h \times K_w$
K_{TILE}	单次 K 分块处理的输入 lane 数
C_{out}	输出通道数
$\text{COUT}_{\text{TILE}}$	单次计算覆盖的输出通道数
H, W, C	特征图高度、宽度、通道数
K_h, K_w	卷积核高度、宽度
oy, ox	输出特征图空间坐标
GEMM	通用矩阵乘法 (GEneral Matrix Multiply)
FPGA	现场可编程门阵列 (Field Programmable Gate Array)
XCK26	Xilinx Kria K26 核心 FPGA 器件型号
WNS	最差负时序裕量 (Worst Negative Slack)
HWC	高度-宽度-通道 (Height-Width-Channel) 内存布局

第一章 绪论

1.1 研究背景与意义

卷积神经网络 (Convolutional Neural Network, CNN) 已经成为目标检测、图像分类和语义理解等视觉任务的核心计算模型。YOLO 系列检测器将候选框生成和类别预测整合为单阶段推理流程, 在边缘端实时感知任务中具有较好的工程适配性^[1-2]。其中 YOLOv3-tiny 通过减少层数和通道规模降低计算量, 适合在嵌入式平台上部署。

边缘端部署面对的约束与数据中心不同。通用 CPU 的吞吐有限, 嵌入式 GPU 或专用 NPU 虽然性能较高, 但在功耗、可定制性、接口灵活性和可验证性方面并不总是满足定制系统需求。FPGA/MPSoC 平台同时包含处理系统 (Processing System, PS) 和可编程逻辑 (Programmable Logic, PL), 能够让软件负责图像输入、调度和后处理, 让硬件承担规则密集的卷积计算。因此, 基于 FPGA 的 CNN 加速器仍然是边缘智能系统中的重要研究方向^[3-4]。

然而, 真实 CNN 网络的端到端加速并不等同于构造一个高峰值 MAC 阵列。以 YOLOv3-tiny 单尺度链路为例, 网络包含 3x3 卷积、1x1 卷积、池化、量化、激活和检测头; 每一层还要经过输入通道展开、输出通道分块、空间 tile 分块和多轮 K pass 累加。若数据搬运、部分和反馈或 pass 边界调度处理不当, 硬件阵列会在等待权重、等待 IFM replay、等待 partial-PSUM drain 或等待 DMA 完成时产生大量空泡。本文关注的核心问题正是: 如何在资源受限的 MPSoC 上, 面向真实 YOLOv3-tiny 链式推理降低非计算开销, 并使优化效果能够被板级实验和性能计数器解释。

1.2 国内外研究现状

CNN 加速器的研究大致经历了从算子级流水线到空间阵列架构的发展。Kung 提出的 systolic architecture 通过规则 PE 网络在相邻单元之间传递数据, 为矩阵乘和卷积计算提供了高复用的硬件组织方式^[5]。Google TPU 使用大规模 systolic array 获得高吞吐推理性能^[6]。Eyeriss 系列则从能量效率角度系统讨论了 row-stationary 数据流和层级存储复用^[7-8]。Sze 等人的综述对 DNN 加速器的数据流、存储层次和能效权衡进行了系统总结^[9]。

在 FPGA 方向, Qiu 等人较早展示了在嵌入式 FPGA 上部署 CNN 的完整流程^[3]; Angel-Eye 进一步提出从模型到 FPGA 映射的设计流程^[4]; S2A 和 RTL compilation 相关工作关注从高层描述生成 systolic array 或模块化 RTL^[10-11]。针对 YOLOv3-tiny 的轻

量化 FPGA 加速器也已有报道^[12]。这些工作说明 FPGA 对 CNN 推理具有可行性，但许多研究更强调模型映射、阵列生成或单层性能，对真实链式推理中的 DMA 交互、IFM 打包、partial-PSUM 反馈和 pass 级调度开销讨论较少。

本文的工作不以绝对 FPS 领先为目标，而是以一个可复现的 MPSoC 端到端系统为载体，研究真实 YOLO 链式推理中的数据流和调度瓶颈。相比仅报告理论吞吐或单层仿真的工作，本文更强调硬件 RTL、Vitis runtime、外部 golden、板级 batch-chain 验证、DDR 图像输入和性能计数器之间的一致闭环。

1.3 本文主要工作

围绕上述问题，本文完成的主要工作如下。

1. **设计并实现面向真实 YOLOv3-tiny 链式推理的卷积加速器架构。**加速器基于 INT8 systolic array，支持 3x3 卷积、原生 1x1 卷积、INT32 partial-PSUM、逐通道重量化、activation LUT、2x2 maxpool 和 OFM 写回；系统接口采用 AXI-Lite 配置和四路 AXI-Stream/DMA 数据通路，能够与 A53 bare-metal runtime 协同执行 Conv0–Conv9。
2. **提出面向真实层的数据流优化方法。**针对频繁 DMA 启动和软件打包开销，本文实现 batch stream 与预打包权重布局；针对后端层 IFM 重排开销，设计 raw-HWC tile cache，使软件传输自然 HWC tile，由硬件按 K pass 和卷积窗口顺序重放；进一步通过 backend full-tile 调度减少 13x13 后端层的空间 tile 固定开销。
3. **优化多 K pass 卷积中的 partial-PSUM 反馈和 pass 级调度。**本文实现流水化 PSUM drain、early drain、continuous PSUM collector、partial-PSUM overlap 和 pass-level prefetch，使当前 pass 的计算、部分和写回、下一 pass 的权重/IFM 准备尽可能重叠，同时保持 AXI 数据格式和软件 ABI 稳定。
4. **建立从 RTL 到 KV260 板级运行的验证和性能诊断体系。**本文使用 Icarus Verilog、Vivado XSIM、外部 RTL golden、链式 semantic golden 和 Vitis board smoke 构成多层验证流程；在硬件中加入 stage、sub-stage、raw-HWC、prefetch、collector、pass timeline 等计数器，用于定位性能瓶颈和解释优化收益。

1.4 论文组织结构

本文共分为六章。第一章介绍研究背景、相关研究现状和本文贡献。第二章给出卷积计算、INT8 量化、systolic array、YOLOv3-tiny 层结构和 MPSoC 平台基础。第三章介绍加速器系统架构，包括 PS/PL 协同、AXI 边界、systolic core、后处理流水线和 OFM 输出路径。第四章讨论数据流与分块调度优化，包括 K/COUT/spatial tiling、batch

stream、预打包权重、raw-HWC cache 和 backend full-tile。第五章介绍 partial-PSUM 反馈、pass-level prefetch、系统集成和性能计数器。第六章给出正确性验证、板级端到端性能、消融实验、资源时序结果和局限性讨论。

第二章 背景与相关工作

本章介绍本文设计所依赖的基础概念，包括卷积计算模型、INT8 量化、systolic array 数据流、YOLOv3-tiny 单尺度网络结构、MPSoC 平台特征以及 FPGA CNN 加速器相关研究。与传统背景介绍不同，本章重点服务于后文的数据流与调度优化：真实网络的性能瓶颈来自层形状、K pass 数量、输出通道分块、空间 tile、部分和反馈和 PS/PL 数据搬运的共同作用。

2.1 卷积计算与分块表示

给定输入特征图 $I \in \mathbb{R}^{H \times W \times C_{in}}$ 、权重 $W \in \mathbb{R}^{K_h \times K_w \times C_{in} \times C_{out}}$ 和偏置 B ，标准二维卷积可以表示为

$$O[y, x, c_o] = B[c_o] + \sum_{c_i=0}^{C_{in}-1} \sum_{k_y=0}^{K_h-1} \sum_{k_x=0}^{K_w-1} I[yS+k_y-P, xS+k_x-P, c_i] \cdot W[k_y, k_x, c_i, c_o], \quad (2.1)$$

其中 S 为 stride， P 为 padding。将卷积窗口和输入通道展开后，可得到 GEMM 形式：

$$O[p, c_o] = B[c_o] + \sum_{k=0}^{K_{total}-1} X[p, k] \cdot W[k, c_o], \quad K_{total} = C_{in} K_h K_w. \quad (2.2)$$

本文硬件按照三个维度组织计算： K 维分块、输出通道分块和空间分块。若 systolic array 每次处理 K_{tile} 个展开通道，则一个卷积层需要

$$N_{kpass} = \left\lceil \frac{K_{total}}{K_{tile}} \right\rceil \quad (2.3)$$

轮 K pass。若每个输出通道块包含 C_{out_tile} 个通道，则输出通道块数为

$$N_{cout} = \left\lceil \frac{C_{out}}{C_{out_tile}} \right\rceil. \quad (2.4)$$

真实层的调度规模由 $N_{tile} \times N_{cout} \times N_{kpass}$ 决定。后端层如 $13 \times 13 \times 512 \rightarrow 13 \times 13 \times 1024$ 虽然空间尺寸较小，但 K_{total} 和 N_{cout} 很大，因此容易暴露 pass 边界和 partial-PSUM 反馈瓶颈。

2.2 INT8 量化与后处理

边缘 CNN 推理通常采用 INT8 量化降低存储带宽和乘加资源消耗。浮点值 r 与量化整数 q 之间可表示为^[13]

$$q = \text{clamp} \left(\text{round} \left(\frac{r}{S} \right) + Z, q_{\min}, q_{\max} \right), \quad (2.5)$$

其中 S 为 scale, Z 为 zero point。卷积内部使用 INT8 输入和权重, 乘加结果累加到 INT32 partial sum。输出时通过逐通道 multiplier、shift 和 output zero point 完成重量化:

$$q_{\text{out}} = \text{clamp} \left(((p\text{sum} \cdot m) \gg s) + Z_{\text{out}}, 0, 255 \right). \quad (2.6)$$

本文加速器将重量化、activation LUT 和可选 maxpool 集成在 PL 端, 从而减少每层输出回到软件后再处理的开销。A53 软件只负责写入每个输出通道 lane 的量化参数和 activation LUT 字节。

2.3 Systolic Array 与数据流

Systolic array 由规则排列的处理单元 (Processing Element, PE) 组成, 数据在相邻 PE 之间按节拍传播, 适合矩阵乘和卷积等规则计算^[5]。不同驻留策略会形成不同数据流: weight-stationary 将权重保持在 PE 内部, input-stationary 保持输入局部性, output-stationary 保持 partial sum, row-stationary 则试图综合利用多类数据复用^[7,9]。

本文采用 weight-stationary 风格。每个 K pass 开始前, 软件或 batch stream 将当前 $K_{\text{tile}} \times C_{\text{out_tile}}$ 权重块送入片上权重存储; 随后 IFM/window 向量流入阵列, PE 执行 INT8 乘法并累加到 INT32 partial sum。该选择的优点是权重加载和阵列计算语义清晰, 便于在 RTL 中实现稳定的 AXI-Stream 边界; 代价是多 K pass 层需要显式 partial-PSUM 反馈路径。

2.4 YOLOv3-tiny 单尺度网络

本文面向的模型是 YOLOv3-tiny 的单尺度链路。输入为 $416 \times 416 \times 3$ 量化 RGB 图像, 硬件链式执行 Conv0 到 Conv9, 最终输出 $13 \times 13 \times 24$ 检测头。表 2.1 给出本文 runtime 使用的层级形状和调度规模。

从表中可以看到, 后端 13x13 层虽然空间尺寸固定, 但 K passes 和 Cout blocks 显著增加, 是本文优化 partial-PSUM、raw-HWC replay 和 pass-level prefetch 的主要目标。

表 2.1 Conv0–Conv9 单尺度链路层参数

层名	IFM	C_{in}	C_{out}	Kernel	K passes	Cout blocks
conv0_pool	416×416	3	16	3x3	2	1
conv1_pool	208×208	16	32	3x3	8	2
conv2_pool	104×104	32	64	3x3	16	4
conv3_pool	52×52	64	128	3x3	32	8
conv4_pool	26×26	128	256	3x3	64	16
conv5	13×13	256	512	3x3	128	32
conv6	13×13	512	1024	3x3	256	64
conv7	13×13	1024	256	1x1	57	16
conv8	13×13	256	512	3x3	128	32
conv9	13×13	512	24	1x1	29	2

2.5 MPSoC 平台与 AXI 接口

Kria KV260 使用 Xilinx Zynq UltraScale+ MPSoC, 包含 ARM Cortex-A53 PS 和 XCK26 PL 逻辑资源。PS/PL 之间通过 AXI4-Lite、AXI4-Stream 和 AXI4-MM 互连进行控制和数据搬运^[14-15]。本文采用 AXI-Lite 作为寄存器配置接口, 并使用四路 AXI DMA: bias、weight、IFM 三路从 DDR 到 PL, OFM 一路从 PL 到 DDR。

AXI-Stream 的 TVALID/TREADY 握手机制有利于处理 PL 内部背压, 但也要求 RTL 对 TLAST、TKEEP、packet count 和 DMA length 做严格约束。本文的 batch stream、raw-HWC cache 和 OFM debug stream 都建立在该边界之上。

2.6 相关工作比较

现有 FPGA CNN 加速器工作在自动化映射、阵列生成、模型压缩和单层吞吐上取得了大量成果^[3-4,10-12]。与这些工作相比, 本文的差异主要在三个方面。

首先, 本文以完整 Conv0–Conv9 链式推理作为验证对象, 而不仅是单层或合成测试。每层输出都需要作为下一层输入, golden 数据也必须遵循同一条 RTL semantic 链路。其次, 本文关注真实 MPSoC 部署中的数据搬运问题, 包含 A53 runtime、AXI DMA、DDR 图像包、OFM packet 重排和 YOLO decode。最后, 本文通过性能计数器对瓶颈迁移进行解释, 强调优化为什么有效以及下一步瓶颈在哪里, 而不是只报告单个最终延迟。

因此, 本文的贡献更接近系统型和数据流优化型研究: 在资源受限的可编程平台上, 将真实 CNN 链路、RTL 数据通路、软件调度和板级性能诊断统一起来。

第三章 加速器系统架构

本章介绍本文加速器的系统结构。设计目标不是单独实现一个卷积算子，而是在 MPSoC 上形成从 DDR 图像输入、PS 侧调度、PL 侧卷积计算、OFM 回写到 YOLO decode 的完整运行链路。为此，硬件架构必须同时满足三个要求：第一，卷积核心具备稳定的 INT8/INT32 计算语义；第二，AXI 边界能够承载真实层的数据量和背压；第三，后处理和输出格式能够支持链式推理和板级验证。

3.1 系统总体结构

图 3.1 给出系统级结构。PS 侧 A53 程序负责准备输入图像包、配置寄存器、启动四路 DMA、维护层和 tile 调度，并在每层结束后解析 OFM packet。PL 侧 accelerator 接收 AXI-Lite 配置，通过 bias、weight、IFM 三路 AXI-Stream 输入完成卷积计算，并通过 OFM AXI-Stream 输出 $\{addr, data\}$ debug packet 到 DDR。

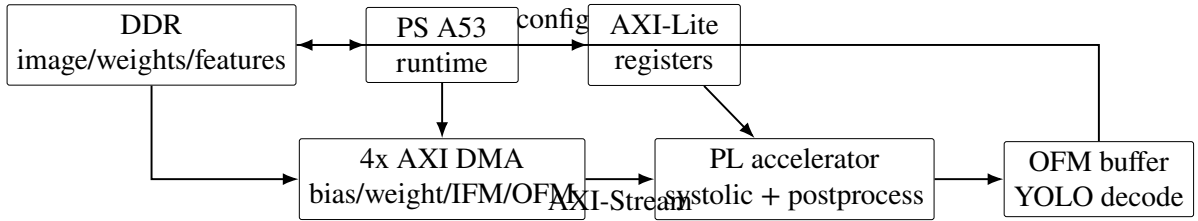


图 3.1 MPSoC 端到端系统结构

控制面和数据面分离是该系统的基本原则。AXI-Lite 寄存器只负责层参数、tile 参数、量化参数、stream 配置和性能计数器；大规模张量数据全部通过 AXI DMA 搬运。这样既避免了 PS 逐字节访问 PL，又使 batch stream、raw-HWC cache 和不同调度开关可以在不改变外部总线结构的情况下演进。

3.2 PL 加速器内部数据流

图 3.2 展示 PL 内部数据流。输入端包含 bias/weight loader、line/vector/raw-HWC IFM loader；中间是 layer scheduler、window feeder、systolic top 和 partial-PSUM 存储；输出端包含 drain/collector、requant、activation、pooling 和 OFM byte writer。

该数据通路采用 packet 化语义。每个输出 pixel 和一个 $Cout_{tile}$ 通道块形成一个 PSUM packet，非 final K pass 写回 partial-PSUM，final K pass 进入后处理流水线。后处理后，每个有效输出 byte 被打包为 OFM debug packet。虽然该输出格式不是最终最高

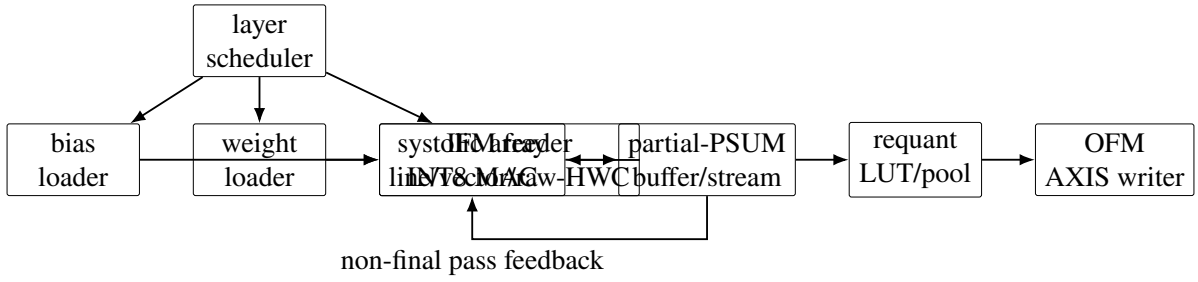


图 3.2 PL 加速器内部数据流

效的连续 HWC burst，但它保留了地址信息，便于软件重排和 golden 比对，是当前板级验证链路的基础。

3.3 Systolic 计算核心

本文最终板级配置采用 ROWS=18、COLS=8、 $C_{out_tile} = 16$ 。阵列沿 K 维接收 18 路 IFM lane，每个列位置对应两个输出通道 lane，因此一个 pass 处理最多 16 个输出通道。PE 内部完成 INT8 乘法和 INT32 累加，权重在当前 pass 内驻留，IFM 向量随输出 pixel 推进。

对 3x3 卷积，IFM feeder 负责根据 oy, ox, k_y, k_x, c_i 生成展开后的 K 维向量。对 1x1 卷积，硬件支持 native vector path，避免把 1x1 强行映射为稀疏 3x3 后带来的无效 K pass。这样 Conv7 和 Conv9 检测头可以使用更短的 K_{total} ，减少后端层调度开销。

3.4 后处理流水线

final K pass 的输出首先进入重量化模块。每个输出通道 lane 有独立的 multiplier、shift 和 output zero point，配置通过 AXI-Lite 间接寄存器写入。重量化后的 uint8 输出进入 activation LUT，支持 identity、ReLU/Leaky ReLU 等运行时可配置语义。若当前层启用 pooling，则 OFM pooling 模块执行 2×2 、stride=2 的 maxpool，并将池化后的输出写入 OFM byte stream。

该设计使 Conv0–Conv4 中的卷积和池化可以在 PL 内连续完成，软件看到的层输出已经是下一层输入所需的 HWC 量化张量。对后端不带 pool 的层，后处理路径直接输出卷积结果。

3.5 AXI-Lite 配置与性能计数器

AXI-Lite 寄存器分为四类。第一类是基础层配置，包括 feature map 尺寸、kernel/stride/pad、 K_{total} 、 C_{out} 、tile 行范围和 expected bytes。第二类是 stream 配置，包括 batch mode、raw-

HWC mode、early drain、pass prefetch、continuous PSUM 和 during-compute prefetch 等开关。第三类是量化和 activation LUT 间接写入寄存器。第四类是只读性能计数器，包括 busy、wait、stage、sub-stage、raw-HWC、prefetch、collector 和 pass timeline 统计。

性能计数器是本文实验论证的重要组成部分。它们不改变外部数据格式，却能在每个 tile 或每层结束后解释加速器时间消耗：等待 DMA、feeder replay、compute stage 空泡、PSUM drain、OFM post-process 分别占多少周期。第六章将使用这些计数器说明优化如何改变瓶颈分布。

3.6 软件运行时协同

Vitis bare-metal runtime 使用静态 layer descriptor 描述 Conv0–Conv9 的输入尺寸、输出通道数、kernel、pool、K passes、Cout blocks 和 golden 信息。运行时按 layer 和 spatial tile 循环，写入配置寄存器，启动 OFM S2MM DMA，再启动输入 MM2S DMA，最后等待 accelerator done 和 OFM DMA 完成。

在验证模式下，软件解析每个 OFM debug packet 的地址和数据，将其重排为 HWC buffer，并与 RTL semantic golden 比较。在 DDR image demo 模式下，软件从固定 DDR 地址读取图像包，执行十层硬件推理后对 Conv9 输出运行 YOLO decode。该路径证明本文设计不仅能通过模块仿真，也能在真实 PS/PL 协同环境中完成端到端图像推理。

第四章 数据流与分块调度优化

本章讨论本文针对真实 YOLOv3-tiny 层提出的数据流与调度优化。若只考虑阵列峰值，卷积层似乎可以由 $MACs/Peak$ 估算延迟；但在 MPSoC 板级运行中，实际时间还包含权重加载、IFM 打包、DMA 启动、tile 边界同步、缓存重放和 OFM 重排。因此，本文将优化目标定义为降低非计算开销：

$$T_{noncomp} = T_{layer} - T_{compute_fire}. \quad (4.1)$$

其中 $T_{compute_fire}$ 表示阵列真正接受一个输出 pixel 计算的周期数，其余部分均属于等待、搬运、反馈或边界开销。

4.1 三维分块调度

本文使用 K 维、输出通道和空间三维分块。K 维分块由 ROWS 决定，最终配置 ROWS=18，因此 $K_{tile} = 18$ 。输出通道分块由 COLS 决定，每列产生两个输出 lane，最终配置 COLS=8，因此 $Cout_{tile} = 16$ 。空间分块由 tile 的输出行范围决定，用于限制片上 IFM/OFM buffer 和单次 AXI DMA 长度。

图 4.1 给出三维调度关系。每个 spatial tile 内，调度器先遍历输出通道块，再对当前输出通道块执行所有 K pass。首个 K pass 从 bias 初始化 partial sum，非首 pass 读取上一轮 partial-PSUM，final pass 输出到后处理流水线。

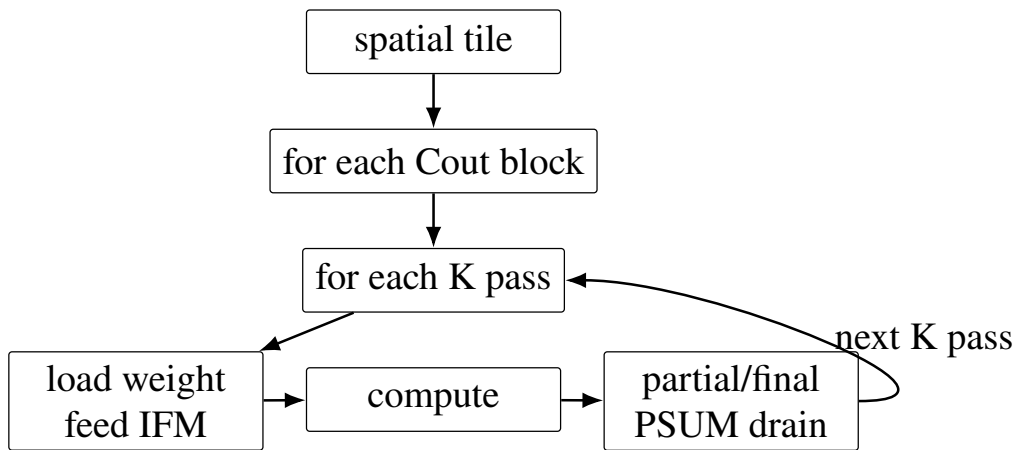


图 4.1 K/Cout/spatial 三维调度关系

该调度方式的优点是通用性强，可覆盖从 Conv0 到 Conv9 的不同层形状；缺点是当 N_{kpass} 和 N_{cout} 较大时，pass 边界开销会被重复放大。后续优化均围绕减少这些重

复开销展开。

4.2 Batch Stream

早期 per-request 模式中，硬件每需要一个 bias block、weight tile 或 IFM line 时向软件发出请求，软件随后打包数据、启动 DMA 并等待完成。这种方式适合小型 smoke test，但在 Conv6 等真实层中会产生数万次服务事件，A53 侧开销和 DMA 启动开销会淹没阵列计算。

Batch stream 将每个 spatial tile 所需的 bias、weight 和 IFM 数据预先打包成连续 AXI stream。软件在 tile 开始时分别启动三路输入 DMA，硬件通过 packet count 恢复内部边界，只在整个 stream 末尾依赖 TLAST。该方式带来两个直接收益：第一，DMA 启动次数从每个请求一次降低为每个 tile 每类输入一次；第二，AXI backpressure 由 PL 自然吸收，软件不需要在 pass 内频繁轮询硬件请求。

STREAM_CFG bit0 控制 batch mode。对应的 BIAS_PACKETS、WEIGHT_PACKETS 和 IFM_PACKETS 寄存器用于配置期望 packet 数，硬件完成后通过只读 counter 回报完成数。软件在每个 tile 结束时比较 expected/completed，保证 stream 边界没有丢包或多包。

4.3 预打包权重布局

权重数据天然以模型格式保存，通常按输出通道、输入通道和 kernel 维度组织；而硬件消费顺序是 *Coutblock* → *Kpass* → *lane* → *channel*。若每次运行时由 A53 重新排列权重，会造成明显的软件时间开销。

本文在构建 Vitis header 时直接生成硬件消费顺序的权重数组。这样 batch mode 下 weight DMA 可以直接读取 ELF 只读段中的连续数组，不再需要运行时 scratch copy 或 repack。PL 端 weight loader 使用 64-bit AXI beat，一拍写入多个 byte bank，进一步降低 weight load 等待。该优化不改变外部 ABI，只改变软件生成数据的排列方式和 PL loader 的内部写入粒度。

4.4 Raw-HWC Tile Cache

IFM 输入存在两种路径。第一种是 prepacked IFM stream：软件提前把每个 K pass 需要的 IFM lane 按硬件顺序打包。该方式 PL 简单，但 A53 侧打包成本较高。第二种是 raw-HWC stream：软件发送自然 HWC tile，PL 在片上缓存后按 pass 和窗口顺序 replay。本文引入 raw-HWC tile cache 以降低后端层的软件打包压力。

图 4.2 对比两种路径。prepacked 路径把重排压力放在软件；raw-HWC 路径把重排压力转移到 PL cache/replay，但可减少 A53 逐 pass 复制和索引开销。对 3x3 层，cache materializes 每个输出 pixel 的 3x3 窗口，并按 18 lane 输出给阵列；对 1x1 层，cache 按连续通道向量 replay。

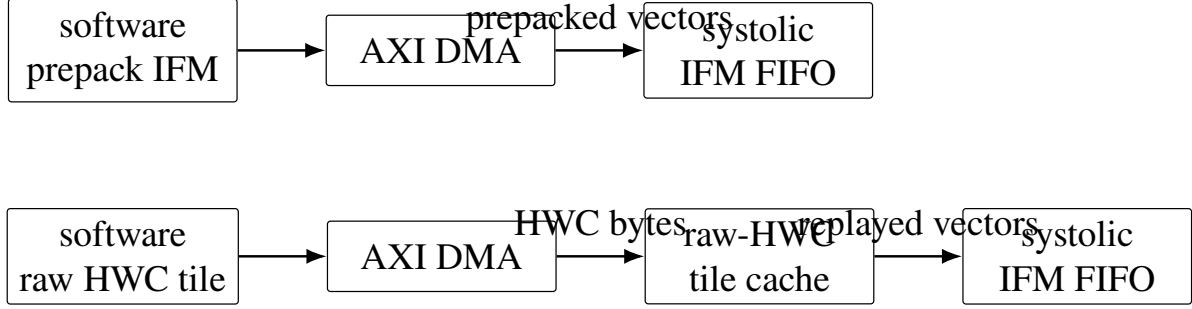


图 4.2 prepacked IFM stream 与 raw-HWC tile cache 对比

raw-HWC 并非对所有层都收益为正。其收益取决于 tile 容量、输入通道数、replay 吞吐和 compute overlap。本文最终将 raw-HWC 作为后端 Conv4/5/6/8 的主要优化路径；Conv3 raw-HWC 虽然可功能通过，但由于 tile/FIFO 容量和 replay 开销，板级结果不优于推荐配置，因此作为边界分析而非默认路径。

4.5 Backend Full-Tile 调度

Conv5、Conv6 和 Conv8 的空间尺寸均为 13×13 。早期调度将这些层拆成 $4+4+4+1$ 行 spatial tile，可以降低片上缓存容量需求，但每个 tile 都需要重复启动 bias、weight、IFM 和 OFM 流程。随着 raw-HWC cache 使用 URAM 扩容，后端层可以使用完整 13×13 tile，从而减少 tile 边界固定成本。

Backend full-tile 的容量条件可近似写为

$$tile_pixels \times \left\lceil \frac{C_{in}}{2} \right\rceil \leq HWC_CACHE_DEPTH. \quad (4.2)$$

在最终 full-tile build 中，Conv5/8 需要 $13 \times 13 \times \lceil 256/2 \rceil = 21632$ 个 materialized word，Conv6 需要 $13 \times 13 \times \lceil 512/2 \rceil = 43264$ 个 materialized word。该容量由 HWC cache 的 URAM 实现承担，换取更少的 spatial tile 启动次数。

4.6 数据流优化小结

本章优化的共同目标是减少 $T_{noncomp}$ 。Batch stream 降低 DMA 启动和软件服务次数；预打包权重消除运行时 weight repack；raw-HWC cache 将 IFM 重排从 A53 移到 PL

并为后续 overlap 创造条件；backend full-tile 降低后端层空间 tile 固定开销。下一章将进一步讨论多 K pass 场景中 partial-PSUM feedback 和 pass-level prefetch 的调度优化。

第五章 Partial-PSUM 反馈与系统集成

多 K pass 卷积的核心难点在于 partial-PSUM feedback。对于 $K_{total} > K_{tile}$ 的层，第一轮 K pass 只能产生部分和，后续 pass 必须读取前一轮 partial sum 并继续累加，最后一轮才进入重量化和输出路径。若每轮 pass 都严格串行执行“计算完成、全部 drain、下一轮 feed、下一轮 compute”，后端大层会出现明显阵列空泡。本章介绍本文如何对 partial-PSUM 路径和 pass 边界进行流水化与重叠。

5.1 Partial-PSUM 反馈语义

对一个输出 pixel 和一个 $Cout_{tile}$ 通道块，硬件内部形成一个 PSUM packet：

$$P[p, c] = \sum_{k=k_0}^{k_0+K_{tile}-1} X[p, k] \cdot W[k, c]. \quad (5.1)$$

当当前 pass 不是 final pass 时，该 packet 被写入片上 partial-PSUM buffer；下一 pass 读取同一地址的 partial sum 作为初值继续累加。当当前 pass 是 final pass 时，该 packet 被送入 requant/activation/pool/OFM path。

该语义要求三个条件同时成立：第一，partial-PSUM 的地址和 Cout block 必须与下一 pass 对齐；第二，写入和读取不能发生读写冲突；第三，final pass 的输出顺序必须与软件重排和 golden 比对一致。本文使用 ping-pong bank 和 per-pass context 维护这些条件。

5.2 流水化 PSUM Drain

早期 drain writer 在下游 ready 出现回压时容易产生地址推进和 valid 保持之间的时序风险，同时吞吐受内部状态机限制。本文将 drain writer 改为 AXI-Stream 风格握手：packet valid、addr 和 data 在真实 *valid&ready* 握手前保持稳定，握手成功后才推进地址。随后加入一拍 read-return tracker 和 one-entry hold register，使下游 ready 时可以接近每周周期输出一个 PSUM packet。

流水化 drain 的意义不仅是提高 drain 吞吐，也为后续 overlap 提供可控事件：调度器可以基于 drain packet fire、drain done 和 FIFO empty wait 判断 partial-PSUM 是否已经产生足够可读数据。

5.3 Early Drain 与 Continuous PSUM Collector

传统 pass 调度在 compute done 后才启动 drain。Early drain 允许调度器在当前 pass 已经开始产生 PSUM 数据后提前启动 drain，只要 partial-PSUM FIFO 已经具备可读数据。该机制降低 compute 完成后等待 drain 启动的空泡，但仍保留 final pass 和非 final pass 的输出顺序约束。

Continuous PSUM collector 进一步改变非 final pass 的路径。collector 在 compute 开始前接收一个 pass context，并在阵列 FIFO 产生完整 packet 后立即将其写入 partial-PSUM bank；final pass packet 则进入后处理流水线。这样，非 final pass 的 partial-PSUM 写回从“compute 后集中 drain”变为“compute 过程中持续收集”，减少 pass 边界处的反馈延迟。

图 5.1 对比了串行 pass 和重叠 pass 的时间线。

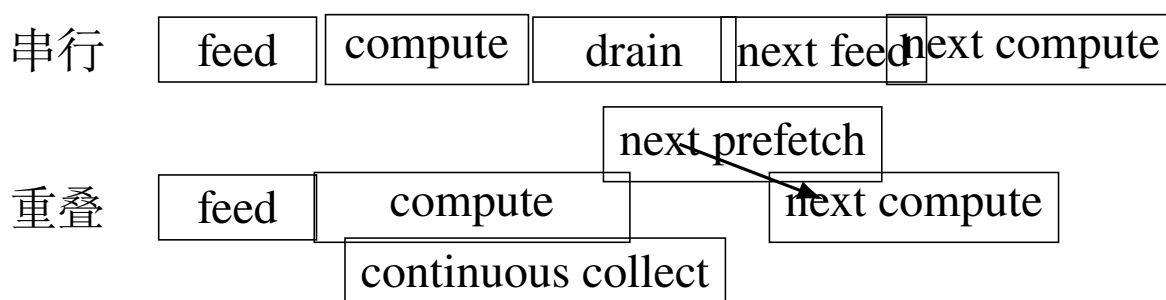


图 5.1 Partial-PSUM 反馈与 pass-level prefetch 时间线

5.4 Pass-Level Prefetch

K pass 之间除了 PSUM 反馈，还包含下一 pass 权重加载和 IFM replay。Pass-level prefetch 在当前 pass 已经满足安全条件后提前准备下一 pass 的 weight 和 raw-HWC replay。该机制遵循两个约束：一是不能覆盖当前 pass 正在使用的 PE 权重；二是 next-pass IFM replay 只能进入解耦 FIFO 或安全队列，不能破坏当前 pass 的输入顺序。

本文实现了两级 prefetch。第一阶段 pass prefetch 主要在当前 compute 完成后、drain 或 pass 边界窗口中准备下一 pass；第二阶段 during-compute prefetch 进一步允许在当前 pass compute 进行时启动下一 pass 权重和 raw-HWC replay 的 staging。后者能够降低 compute stage 中由于下一 pass 准备不足造成的 idle，但需要更严格地维护权重读取预算和 IFM FIFO 顺序。

5.5 STREAM_CFG 与调度开关

所有优化均通过 AXI-Lite 中的 STREAM_CFG 位控制。bit0 使能 batch mode, bit1 使能 raw-HWC mode, bit2 使能 early drain, bit3 使能 pass prefetch, bit4 使能 partial-PSUM overlap, bit5 使能 continuous PSUM collector, bit7 使能 during-compute prefetch。bit6 对应 column-level PSUM, 是当前探索性设计, 不作为最终推荐配置的必要条件。

这种设计保持了软件 ABI 的稳定性。默认关闭实验开关时, 系统回到保守串行语义; 逐步打开开关时, 软件只需写入不同 STREAM_CFG 和少量 tail/prefetch 参数, 外部 AXI packet 格式保持不变。该策略便于在同一 runtime 中进行 A/B 测试和板级回归。

5.6 性能计数器设计

为了避免仅凭端到端延迟判断优化效果, 本文在硬件中加入多层性能计数器。

表 5.1 主要性能计数器类别

类别	作用
PERF	统计 tile busy、外部等待、compute fire 等顶层周期
STAGEPERF	统计 bias、weight、feeder、compute、drain、OFM post 阶段
SUBPERF	细分 feeder push、FIFO stall、compute active、IFM stall、tail 等
RAWSTAT	统计 raw-HWC load、unpack、replay 和 replay wait
PREFETCHPERF	统计 prefetch start、hit、miss、stall 等事件
COLLECTPERF	统计 collector packet、partial write、final write 和 context stall
PASSPERF	统计 pass start 到 first fire、fire span、compute idle 等

这些计数器使本文能够说明每次优化后的瓶颈迁移。例如, drainpipe 主要降低 drain 阶段; continuous collector 降低 partial-PSUM 存储开销; backend full-tile 减少 spatial tile 固定成本; during-compute prefetch 降低下一 pass 准备造成的 compute idle。第六章将基于这些计数器进行实验分析。

5.7 探索性 Column-Level PSUM

当前 continuous PSUM collector 仍以完整 packet 为反馈粒度: collector 等待所有列 FIFO 都有数据后写入一个 $COLS \times 2$ 通道 packet。Column-level PSUM 的探索方向是按列独立写入和读取 partial sum, 使下一 pass 能够按照 systolic array 的列传播延迟更早获得可用数据。

本文已经实现了 column-level ping-pong buffer 和 column stream feeder 的基础模块, 并完成模块级验证。但该路径尚未作为最终推荐板级配置, 因此本文将其作为未来工

作讨论，不将其计入核心性能贡献。这样可以避免把尚未完整端到端验证的实验性设计包装为成熟成果。

5.8 本章小结

本章从 partial-PSUM 反馈角度解释了后端大层的性能瓶颈，并介绍了 drainpipe、early drain、continuous collector、pass prefetch 和 during-compute prefetch 等调度优化。与第四章的数据搬运优化结合后，系统能够同时减少软件打包、DMA 启动、tile 边界、PSUM drain 和 pass 准备空泡，为第六章的端到端性能提升奠定基础。

第六章 验证与实验结果

本章报告本文加速器的正确性验证、板级端到端性能、优化消融、阶段计数器分析和资源时序结果。实验的重点不是将最终延迟包装为实时视频领先指标，而是验证本文提出的数据流和调度优化是否能够在真实 MPSoC 环境中降低 YOLOv3-tiny 链式推理的非计算开销，并解释性能瓶颈如何迁移。

6.1 实验设置

实验平台为 Xilinx Kria KV260，PL 器件为 XCK26，工具链为 Vivado/Vitis 2022.2。最终推荐硬件构建目录为 D:/MPSoC/b_kprefetch_22，核心时钟目标为 100 MHz。加速器参数为 ROWS=18、COLS=8、 $C_{out_tile} = 16$ 、IFM FIFO depth=1024，后端 raw-HWC full-tile cache 使用 URAM 承载 Conv5/6/8 的 13×13 tile。

最终推荐软件配置为 conv0_conv9_ddr_demo，开启 RawHwcConv4/5/6/8、early drain、pass prefetch、partial-PSUM overlap、continuous PSUM、backend full-tile 和 during-compute prefetch。验证路径包含两类：第一类是 batch-chain 模式，对 Conv0–Conv9 每层输出与 RTL semantic golden 做 bit-exact 比较；第二类是 DDR image demo，将 $416 \times 416 \times 3$ 图像包写入 DDR，由硬件执行十层推理并在 A53 侧进行 YOLO decode。

6.2 正确性验证

本文采用从模块到板级的多层验证。RTL 层面使用 Icarus Verilog 和 Vivado XSIM 覆盖 PE、array、feeder、AXI loader、PSUM buffer、collector、AXI-Lite bridge 和端到端 conv top。系统层面使用外部 golden 数据验证真实 Conv0、Layer06/Conv3、Conv4 以及后端 Conv5/6/8 raw-HWC tile。板级层面使用 KV260 batch-chain 和 DDR image demo 验证完整链路。

需要强调的是，链式 golden 与单层 standalone golden 不可混用。由于前级硬件量化、激活和池化语义会影响后级输入，Conv5 之后的 expected output 必须由同一条 RTL semantic chain 生成。本文在 batch-chain 中逐层保存硬件输出并比较，从而验证层间数据重排、feature buffer 切换和检测头输出的一致性。

表 6.1 关键正确性验证结果

验证对象	覆盖内容	结果
Conv0 crop + pool	真实输入 crop、卷积、LUT、pool、OFM packet	0 mismatch
Layer06/conv3 pool	$52 \times 52 \times 64 \rightarrow 52 \times 52 \times 128$	XSIM 与板级通过
Conv4 pool	$26 \times 26 \times 128 \rightarrow 13 \times 13 \times 256$	XSIM 与板级通过
Conv5/6/8 raw-HWC Conv0–Conv9 batch-chain	后端 full-tile raw-HWC + overlap 配置 十层链式 RTL semantic golden	XSIM 2726/0 PASS
DDR image demo	动态图像输入与 YOLO decode	两张图像 PASS

6.3 端到端性能与优化消融

表 6.2 汇总了主要板级优化版本的端到端 DDR demo 延迟。所有结果均来自 KV260 板级运行，输入图像和模型链路保持一致。不同版本的目标不是形成严格产品发布序列，而是展示本文优化如何逐步降低非计算开销。

表 6.2 端到端 DDR demo 优化消融

版本或优化阶段	主要变化	典型延迟
Batch/DDR baseline	batch stream, DDR 图像 demo 初版	≈ 1.18 s
Weight 64-bit loader	预打包权重与 64-bit PL 解包	≈ 0.861 s
Drainpipe	流水化 PSUM drain writer	≈ 0.646 s
Early drain + pass prefetch	提前 drain 与下一 K pass 预取	≈ 0.387 s
Continuous PSUM collector	非 final pass 持续写 partial-PSUM	≈ 0.371 s
Backend full-tile	Conv5/6/8 使用 13×13 full tile	≈ 0.335 s
During-compute prefetch	当前 compute 中准备下一 K pass	≈ 0.288 s

最终 b_kprefetch_22 构建完成 full-programming batch-chain 验证，日志中 UART detections 与 decode golden 匹配。两张 DDR image demo 的结果如表 6.3 所示。

表 6.3 最终 DDR image demo 结果

图像	总延迟	检测结果	日志
maksssksksss0.png	288.002 ms	with_mask, score=0.357321	20260616_193623
maksssksksss1.png	287.993 ms	with_mask, score=0.295050	20260616_193752

该结果约为 3.5 FPS，不应表述为通用实时检测领先性能。本文将其解释为在资源

受限 XCK26 上，通过数据流与调度优化将早期约 1.18 s 的完整链式推理降低到约 0.29 s，获得约 4.1 倍端到端加速。

6.4 阶段计数器分析

仅报告端到端延迟无法说明优化来源。表 6.4 给出最终 DDR demo 的关键硬件计数器换算时间。可以看到，最终硬件 busy 时间约为 247.184 ms，端到端 total 与 PL busy 之间的差额来自 A53 runtime、DMA 管理、cache maintenance、OFM 重排和 YOLO decode 等软件侧开销。

表 6.4 最终版本关键阶段计数器

指标	时间
Total DDR demo	288.002 ms
HW busy	247.184 ms
compute_fire	74.323 ms
feeder stage	99.913 ms
compute stage	121.022 ms
compute idle in stage	46.699 ms
drain stage	16.467 ms

这些计数器说明，经过 drainpipe 和 continuous PSUM 后，drain 已不再是主要瓶颈；经过 backend full-tile 和 during-compute prefetch 后，剩余开销主要分布在 feeder/replay 和 compute stage idle。换言之，下一步优化不应继续单纯扩大阵列，而应继续提高 raw-HWC replay 吞吐、减少 compute stage 内部空泡，并评估 OFM debug packet 对软件重排的影响。

6.5 资源与时序

表 6.5 给出最终推荐硬件构建的实现资源与时序。该版本在 Vivado 2022.2 下完成布局布线，WNS 为正，说明 100 MHz 目标可闭合。

资源结果体现了本文优化的代价。raw-HWC full-tile cache 引入 URAM 以减少后端 tile 启动开销；continuous PSUM collector 和性能计数器增加 LUT/FF；during-compute prefetch 增加调度与队列控制逻辑。整体上，设计仍能装入 XCK26 并闭合 100 MHz，但 LUT 使用已经较高，继续增加 column-level PSUM 或更深 overlap 时需要谨慎评估资源和时序裕量。

表 6.5 最终推荐构建资源与时序

项目	数值
Build directory	D:/MPSoC/b_kprefetch_22
CLB LUTs	84480
CLB Registers	53260
BRAM Tile	63
URAM	24
DSP	184
WNS	+0.092 ns
TNS	0 ns
WHS	+0.010 ns
Routing errors	0

6.6 与相关工作的定性比较

表 6.6 从研究重点角度比较本文与代表性工作。由于不同论文使用的模型、输入尺寸、器件、频率、量化策略和是否计入软件后处理差异较大，本文不进行不公平的绝对 FPS 横向排名。

表 6.6 与代表性工作的定性比较

特征	Angel-Eye ^[4]	S2A ^[10]	本文
主要目标	自动化映射流程	systolic array 生成	MPSoC 链式推理优化
网络粒度	多模型映射	阵列综合	YOLOv3-tiny Conv0–Conv9
数据流讨论	编译映射为主	阵列结构为主	DMA/IFM/PSUM/pass 调度
板级端到端	有	非重点	KV260 DDR image demo
性能解释	较少	较少	stage/pass/counter 诊断

本文的优势在于系统闭环和瓶颈解释；不足在于通用性和绝对吞吐仍有限。当前 runtime 针对 YOLOv3-tiny 单尺度静态调度，不是通用 CNN 编译器；OFM 输出仍采用 debug packet 格式，验证友好但带宽效率不足；功耗数据尚未形成板级实测，后续可结合 Vivado power estimate 或外部功耗计进行能效评估。

6.7 局限性与后续工作

本文仍存在以下局限。第一，最终延迟约 288 ms，不能宣称实时视频级性能领先。其价值在于证明优化路径和系统完整性，而不是与 GPU/NPU 进行绝对性能竞争。第二，OFM debug packet 以每字节一个带地址 packet 的方式输出，便于验证但降低 DMA 带宽效率；后续应设计连续 HWC burst 写回。第三，raw-HWC cache 对层形状敏感，Conv3 实验说明并非所有层都适合启用该路径。第四，column-level PSUM 仍处于探索阶段，需要完整 top-level xsim 和板级 A/B 后才能作为正式优化。

后续工作可从四个方向展开：一是将 OFM 输出迁移为连续 HWC DMA burst，减少软件重排；二是继续优化 raw-HWC replay 吞吐和 compute stage idle；三是完善 column-level PSUM feedback，使 partial sum 按列粒度更早进入下一 pass；四是补充功耗实测和更广泛模型验证，提升论文结论的可比性和可推广性。

6.8 本章小结

本章给出了本文加速器的正确性、性能、资源和限制分析。实验表明，本文设计能够在 KV260/XCK26 上完成真实 Conv0–Conv9 链式推理，并通过 batch stream、raw-HWC cache、backend full-tile、continuous PSUM 和 during-compute prefetch 将端到端 DDR demo 延迟降低到约 0.29 s。性能计数器进一步说明，优化后瓶颈已从 PSUM drain 和权重加载转移到 feeder/replay 与 compute idle，为后续工作提供了明确方向。

参考文献

- [1] REDMON J, DIVVALA S, GIRSHICK R, You only look once: Unified, real-time object detection// Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788.
- [2] REDMON J FARHADI A. YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767, 2018.
- [3] QIU J, WANG J, YAO S, Going deeper with embedded FPGA platform for convolutional neural network//Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2016: 26-35.
- [4] GUO K, SUI L, QIU J, Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2019, 37(1): 35-47.
- [5] KUNG H T. Why systolic architectures?//Computer: vol. 15: 1. 1982: 37-46.
- [6] JOUPPI N P, YOUNG C, PATIL N, In-datacenter performance analysis of a tensor processing unit// Proceedings of the 44th Annual International Symposium on Computer Architecture. 2017: 1-12.
- [7] CHEN Y H, KRISHNA T, EMER J S, Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. ACM SIGARCH Computer Architecture News, 2016, 44(3): 367-379.
- [8] CHEN Y H, YANG T J, EMER J, Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 2019, 9(2): 292-308.
- [9] SZE V, CHEN Y H, YANG T J, Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE, 2017, 105(12): 2295-2329.
- [10] WEI X, YU C H, ZHANG P, Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs//Proceedings of the 54th Annual Design Automation Conference. 2017: 1-6.
- [11] MA Y, CAO Y, VRUDHULA S, Scalable and modularized RTL compilation of convolutional neural networks onto FPGA//2018 28th International Conference on Field Programmable Logic and Applications. 2018: 1-8.
- [12] DING C, WANG S, LIU L, A lightweight FPGA-based CNN accelerator design for YOLOv3-tiny. Microelectronics Journal, 2022, 120: 105344.
- [13] JACOB B, KLIGYS S, CHEN B, Quantization and training of neural networks for efficient integer-arithmetic-only inference. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018: 2704-2713.
- [14] Arm Ltd.AMBA AXI and ACE Protocol Specification. Arm Ltd., 2021.
- [15] Xilinx Inc.Zynq UltraScale+ MPSoC Data Sheet: DC and AC Switching Characteristics. Xilinx Inc., 2022.